EC3290 – Software Requirements Engineering

Topic 5 – Advanced Topics in Software Requirement Engineering

Topics:

- Hazard Analysis & Safety-Critical Requirements
- Threat Modeling & Security-Critical Requirements
- Tool Support in Requirements Engineering
- Open Research Challenges

Introduction to Advanced Topics

- As modern software systems become increasingly embedded in critical environments—from healthcare and transportation to finance and defense—the field of requirements engineering must evolve. Four advanced areas that reflect the growing demand for dependable, secure, and maintainable systems.

Hazard Analysis and Safety-Critical System Requirements

Safety-critical systems are those where failures can result in catastrophic consequences: human injury, death, or major financial loss. Think of pacemakers, aircraft autopilot systems, or nuclear power controls.

The goal of requirements engineering in such systems is not just to ensure functionality—but to safeguard against harmful scenarios.

Hazard Analysis?

Hazard analysis is the structured process of identifying conditions or events that could lead to system failure or harm.
It helps us answer:

- What can go wrong?
- What are the consequences?
- What measures can prevent it?

This early-stage analysis directly influences how safety requirements are written.

Techniques in Hazard Analysis

Three popular techniques are widely used in industry:

- **FMEA** (Failure Modes and Effects Analysis): Assesses how different parts might fail and the impact.
- **FTA** (Fault Tree Analysis): A top-down diagram-based method to analyze cause-and-effect.
- **HAZOP** (Hazard and Operability): A systematic team review to find deviations from intended operations.

Failure Modes and Effects Analysis (FMEA)

FMEA is a **bottom-up** approach that identifies potential failure modes of each component, analyzes their consequences, and prioritizes them based on severity, occurrence, and detectability.

Steps in FMEA:

1. List components and functions
2. Identify how each can fail (failure modes)
3. Analyze effects of each failure
4. Assign Risk Priority Number (RPN)
5. Recommend corrective actions

Failure Modes and Effects Analysis (FMEA)

Example:

In a **heart monitor** system:

- Component: ECG sensor
- Failure Mode: Sensor disconnect
- Effect: No reading → risk to patient
- RPN: High → Add alert system when no signal detected

Fault Tree Analysis (FTA)

FTA is a **top-down** graphical technique that starts from an undesired event and uses logic gates (AND, OR) to trace the possible root causes.

Steps in FTA:

1. Define the undesired "top event"
2. Break down causes using logic gates
3. Identify minimal cut sets (combinations that can trigger the top event)
4. Analyze probabilities and dependencies

Fault Tree Analysis (FTA)

Example:

Top event: “Aircraft loses power”

- Cause A: Fuel exhaustion
- Cause B: Engine control software crash
  FTA builds a visual tree showing how these causes interact.

Hazard and Operability Study (HAZOP)

HAZOP is a **structured team-based** technique that analyzes potential deviations from the intended design using guide words like "more," "less," "none," etc.

Steps in HAZOP:

1. Form a multidisciplinary team
2. Divide system into nodes/processes
3. Apply guide words to each node (e.g., "No flow", "More pressure")
4. Brainstorm possible causes and consequences
5. Document actions needed

Hazard and Operability Study (HAZOP)

In a **chemical plant**:

- Node: Pipeline transporting gas
- Guide word: “More pressure”
- Deviation: Pressure above safety threshold
- Consequence: Explosion risk
- Action: Add pressure relief valve

Comparison of Hazard Analysis Techniques

Technique	Approach	Focus	Best For	Strengths	Limitations
FMEA	Bottom-up	Component-level failures	Mechanical/electrical systems	Simple to apply, quantifies risk	May miss system-level interactions
FTA	Top-down	Undesired events & causes	Aerospace, nuclear, automotive	Clear visualization of failure logic	Complex for large systems
HAZOP	Team-based with guide words	Process deviations	Chemical/process industries	Encourages brainstorming, very thorough	Time-consuming, needs expertise

From Hazards to Requirements

Once hazards are known, we specify how the system should respond or prevent these situations.
For example:

- Hazard: Sensor overheating
- Requirement: “The system shall shut down the module if temperature exceeds 80°C.”

These are **non-functional requirements**, often focusing on system behavior under stress or failure.

Threat Modeling and Security-Critical Requirements

- Why Security Matters
  - Security is now an essential requirement in almost every system. From personal apps to critical infrastructure, breaches can expose private data or even endanger lives.
  - Unlike hazards (accidental), threats are **intentional**, posed by attackers. Our role is to anticipate and mitigate them through proper requirements.

Threat Modeling

Threat modeling is a proactive approach to uncovering and analyzing security threats during the design phase.

- Steps include:

1. Identifying assets (data, user credentials, etc.)
2. Listing potential attackers and their motives
3. Pinpointing vulnerabilities
4. Defining countermeasures

STRIDE Framework

The STRIDE model categorizes six types of threats:

- **Spoofing:** Impersonating identities
- **Tampering:** Modifying data
- **Repudiation:** Denying actions
- **Information Disclosure:** Leaking data
- **Denial of Service:** Disrupting services
- **Elevation of Privilege:** Gaining unauthorized access

For each threat type, we derive one or more security requirements.

S – Spoofing Identity

What is Spoofing?
When someone pretends to be someone else to gain unauthorized access.

Example:

- A hacker logs into a system using someone else's username and password.

Analogy:

Like someone using a fake ID to enter a restricted event pretending to be someone else.

Real-world Scenario:

- An attacker uses a phishing website that looks like a real bank site to trick users into logging in, then steals their credentials.

Typical Requirement to Prevent:

- "The system shall enforce multi-factor authentication for all user logins."

T – Tampering with Data

What is Tampering?
When someone changes data without permission.

Example:

- Modifying a software file to insert malicious code.
- Changing the amount in a bank transfer during transmission.

Analogy:

Like altering a signed document to change the contract terms.

Real-world Scenario:

- Hackers inject malicious scripts into a website (e.g., form hijacking in e-commerce sites).

Typical Requirement to Prevent:

"All transmitted data shall be encrypted and validated upon receipt."

R – Repudiation

What is Repudiation?
When a user **denies** having performed an action, and there's **no proof** to say otherwise.

Example:

- A user deletes records and claims they didn't do it.

Analogy:

Imagine someone claiming, "I didn't send that email," and you can't prove otherwise.

Real-world Scenario:

- A system with no logging mechanism allows users to delete files without tracking who did it.

Typical Requirement to Prevent:

- "The system shall maintain detailed and tamper-proof logs of all user actions."

I – Information Disclosure

What is Information Disclosure?
When sensitive information is exposed to someone who shouldn't have access to it.

Example:

- A website mistakenly shows other users' account info.

Analogy:

Like a bank teller shouting out your account balance in public.

Real-world Scenario:

- A developer forgets to turn off debug mode, and sensitive data is shown in browser error messages.

Typical Requirement to Prevent:

- "The system shall ensure access controls are applied to all confidential data."

D – Denial of Service (DoS)

What is Denial of Service?
When someone makes a system **unavailable** to others by overwhelming it.

Example:

- A flood of traffic crashes a website so real users can’t access it.

Analogy:

Like filling up all the seats in a theater with fake bookings so real people can’t attend.

Real-world Scenario:

- Bot attacks on an online store during a sale to block other buyers.

Typical Requirement to Prevent:

- “The system shall detect and limit excessive requests

E – Elevation of Privilege

What is Elevation of Privilege?
When someone gains more access or power than they're supposed to have.

Example:

- A regular user finds a flaw and becomes an admin.

Analogy:

Imagine a junior staff member finding the keys to the CEO's office and taking over.

Real-world Scenario:

- Exploiting a bug to bypass role checks and access restricted parts of the system.

Typical Requirement to Prevent:

- "The system shall enforce role-based access control for all operations."

STRIDE Summary Table

Threat	Description	Example	Requirement
Spoofing	Pretending to be someone else	Fake login credentials	Use two-factor authentication
Tampering	Changing data or files	Modify code during update	Use encryption and validation
Repudiation	Denying actions without proof	Delete logs, deny wrongdoing	Use audit logs
Info Disclosure	Exposing sensitive info	Leak user data in error	Apply access control
DoS	Overloading systems	Attack to crash website	Use rate-limiting
EoP	Gaining extra access	Regular user becomes admin	Role-based access control

Security-Critical Requirements Example

Imagine a mobile banking app:

- **Threat:** Credential theft
- **Requirement:** "The system shall enforce biometric login for all high-value transactions."

Good security requirements must be **clear, testable, and directly address identified threats.**

Tool Support for Requirements Engineering

As systems and requirements grow in number and complexity, manual tracking becomes inefficient and risky.
Tools offer support for:

- Requirement gathering
- Traceability
- Change management
- Consistency checking

Categories of Requirements Tools

1. **Requirements Management Tools**: e.g., IBM DOORS, Jama Connect
2. **Modeling Tools**: e.g., Enterprise Architect
3. **Traceability Tools**: e.g., Reqtify
4. **Formal Verification Tools**: e.g., Alloy, SPIN

- Each tool specializes in a phase of the requirements lifecycle—from elicitation to validation.

Key Benefits of Tools

Using tools can help:

- Ensure **requirements traceability** (from user needs to code)
- Avoid **duplication or conflicts**
- Manage **requirements evolution** over time
- Facilitate **team collaboration**
- Enable **regulatory compliance** in safety/security-critical projects

Example Tool Workflow

A typical workflow could look like this:

1. Define requirements in IBM DOORS
2. Model interactions in Enterprise Architect
3. Link tests in JIRA
4. Maintain traceability with Reqtify
   This ensures consistency and confidence that all requirements are being fulfilled.

Open Research Problems in Requirements Engineering

Today's systems are **adaptive**, **intelligent**, and operate in **real-time** environments. Static requirements documents cannot capture their fluid behavior.

This evolution opens up several research problems that scholars and engineers are actively working on.

Major Open Research Problems

- **Dynamic Requirements in Agile**
  How to trace and validate rapidly evolving requirements?
- **Ambiguity in Natural Language**
  Can we auto-detect vague terms like “fast,” “secure,” or “user-friendly”?
- **AI and Learning Systems**
  How do we define “correct behavior” in systems that evolve?
- **Ethics and Responsibility**
  How do we encode social values (e.g., fairness, explainability)?
- **Automation in Requirements Elicitation**

Future Directions and Innovations

- **NLP** for automated requirement classification
- **Ontologies** for domain-specific vocabularies
- **Crowdsourcing** to collect public requirements
- **Digital Twins** to simulate and validate requirements in real-time

Summary

We’ve explored:

- How **hazard analysis** supports safety in critical systems
- How **threat modeling** anticipates malicious attacks
- How **tools** make requirement management scalable
- The **future** of requirements engineering in AI, ethics, and automation

As software continues to shape our world, the way we **define, manage, and validate requirements** must evolve too.